

Shipping Network Optimization with Reinforcement Learning: A Simple Example

Team members

- **Leaders**

1. **Hong Ming Tan:** Senior Lecturer, Deputy head of DAO
2. **Qinghe Sun:** Assistant Professor of Hong Kong Polytechnic University in Maritime Logistics, Supply Chain Resilience
3. **Jussi Keppo:** Head of DAO and Professor of IORA

- **Full-time team members**

1. **Xuefei Liu:** Staff of IORA
2. **Linsheng Zhuang:** PhD student of IORA

0. Motivation

Two targets

1. Given the current ports and lines, how to make adjustments to reduce the total operations cost (**Line Optimization**)
2. Given the lines, how to allocate vessels, schedule the visiting time, and allocate cargos. (**Link to the Applications**)

Some Observations

- Line Optimization is a **sequential decision**.

Trajectory: $S_0, a_1 \in \mathcal{A}_1(S_0), S_1, a_2 \in \mathcal{A}(S_1), \dots$

- Why?

Curse of dimensionality

- Reinforcement learning? AlphaGo?

Not really applicable: the mind is too "deep"...

- Our algorithm is a myopic version of AlphaZero

1. Prototype

The purpose of this project is to give the best "adjustment" of the current network, including lines and the arrangements of ships (amount and calendar).

Due to the complexity, we divide the problem into 4 layers:

- In layer 0, we solve an approximated cost function given lines and weekly demand rate.
(for Target 1)
- In layer 1, we focus only on the line generation problem given the approximated cost function.
(for Target 1)
- In layer 2, we build NN to learn the optimal service graph adjustment policy.
- In layer 3, we solve the remaining demand fulfillment problems and scheduling problem of ships given the generated lines.

(for Target 2)

Layer 0: Approximate the minimum cost

The purpose of layer 0 is to provide an **approximated** cost function for layer 1, given the network of lines and weekly demand rate as input.

The key problem of this layer to be solved is the **cargo allocation problem** (or "demand fulfillment problem"). We don't concern more detailed problems like how many vessels to be assigned to each line, and the calendar of each ship. Instead, we only consider the **average capacity** of each line of a week, using the average demand of a week.

The optimization problem of layer 0 can be abstractly represented by

$$\text{Cost}(G) = \text{the sum of} \begin{cases} \text{chartering cost,} \\ \text{bunkering cost,} \\ \text{transshipment cost,} \\ \text{port call cost} \end{cases} \quad (1)$$

subject to

- The average demand of a week are all fulfilled.

We will estimate all these costs with the simplest measurement on **line level**.

Layer 1: Service Line design

The purpose of Layer 1 is to design lines.

The optimization problem of layer 1 can be abstractly represented by

$$\min_{\text{actions}} \text{Cost}(G) \quad (2)$$

where $G = \{T1, T2, \dots\}$ means network of lines, and **actions** includes

- Adding ports into lines
- Removing ports from lines
- Creating new lines

The **Cost** function is derived by layer 0. Here, we just take it as a given blackbox.

This is a network optimization problem. In traditional OR, people typically solve this problem by **MILP**. However, for such a complex problem, mixed integer optimization programming algorithm is usually intractable. Instead, people have to either simplify their model or resort to some heuristic method to simplify the calculation.

We solve the problem by Monte Carlo Tree Searching (MCTS) algorithm.

Layer 2: Neural Network

Based on the above Monte Carlo Tree, we build NN to learn the optimal adjustment policy for any given lines.

(The coding part is done already. We just need some resources to train the neural network.)

Layer 3: Ship Level Decision

Layer 2 is a finer version of layer 0. Remember that in Layer 0, we only measure the efficiency of the line using average demand rates and capacity of whole line.

Here, we move on to the vessel level, solving the

- Demand Fulfillment Problem
 - Allocation of demands
 - Arrangement of ships: how many ships (of each classes) are arranged to each line
- Scheduling Problem of Each Vessel
 - The speed of ships on the whole line
 - Calendar

The main difference of Layer 2 and Layer 0 is the **granularity** of the problem. In layer 0, we only consider the line level optimization, but here we consider ship level optimization.

The optimization problem of layer 2 is still represented by

$$\text{Cost}(G) \quad \text{includes} \quad \left\{ \begin{array}{l} \text{chartering cost,} \\ \text{bunkering cost,} \\ \text{transshipment cost,} \\ \text{port call cost} \end{array} \right. \quad (3)$$

subject to

- Demands are all fulfilled
- Vessel call a port on the same day (of week)
- Others...

2. Atoms

In this section, we define the notations of this model. These are the basis of this huge model.

2.1. Port

In Python code, all ports related class and methods are defined in `port.py` file. There are three classes

- `Port`: containing attributes of each port
- `PortsPool`: the collection of all ports that we consider in this model
- `PortGraph`: the mutual relation (distance, demand) among ports.

(1) The Pool of All Ports

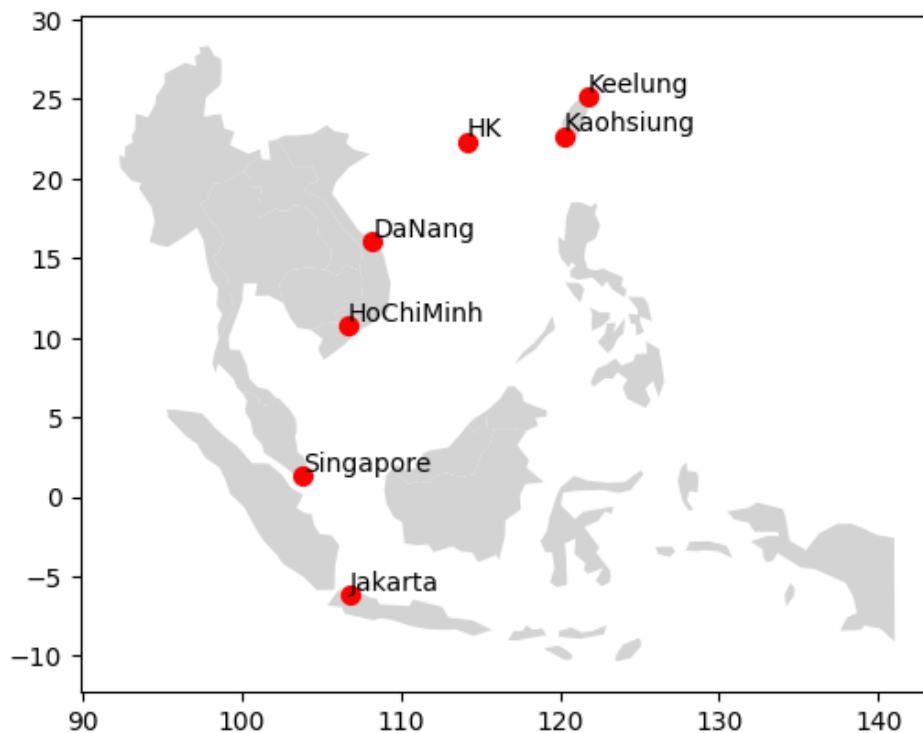
In the simple example, we consider 7 ports.

The pool of all ports is denoted by $\mathcal{I} = \{P1, P2, P3, P4, P5, P6, P7\}$.

These 7 ports are

- P1(HK)
- P2(DaNang)
- P3(HoChiMinh)
- P4(Singapore)
- P5(Jakarta)

- P6(Kaohsiung)
- P7(Keelung)



(2) Port Attributes: P1(HK) as an example

- ID: P1
- Name: Hongkong
- Location (longitude, latitude): (114.177216, 22.3193).
- Max Port Draft
 - For simplicity, we just assume that all port draft are **infinity**.
- Transshipment cost and time per TEU of goods.
 - Port "P1" transships 1 TEU of goods at cost 10000 in one day.
- Port call cost for each vessel class per time unit.
 - For example, port "P1" allows for vessel classes 1, 2, 3 to call with cost 10000, 20000, 30000 per day.
 - We can calculate an **average** port call cost rate to eliminate classes.

(3) Additional Information on Ports

- Mutually Relative Distances: (Source: <http://ports.com/sea-route> and <https://www.prokerala.com/travel/distance/>)

	P1	P2	P3	P4	P5	P6	P7
P1	-	667	1065	1396	1762	414	590
P2	667	-	506	1273	1327	1043	1240
P3	1065	506	-	775	1017	1432	1629
P4	1396	1273	775	-	482	2126	2314
P5	1762	775	1017	482	-	1894	2531
P6	414	1017	1432	2126	1894	-	232
P7	590	482	1629	2314	2531	232	-

Notation: $m^{(i,j)}$.

- Mutual Demand Flow in a Week (I made up the data, unit: TEU)

	P1 (to)	P2 (to)	P3 (to)	P4 (to)	P5 (to)	P6 (to)	P7 (to)
P1 (from)	-	520	660	350	780	390	440
P2 (from)	1030	-	2043	170	498	540	532
P3 (from)	2300	1890	-	260	339	270	709
P4 (from)	650	650	890	-	587	358	375
P5 (from)	820	279	421	390	-	576	299
P6 (from)	460	345	120	260	425	-	804
P7 (from)	680	301	290	180	201	1023	-

Notation: $D^{(o,d)}$.

2.2. Service

(1) Service Attributes

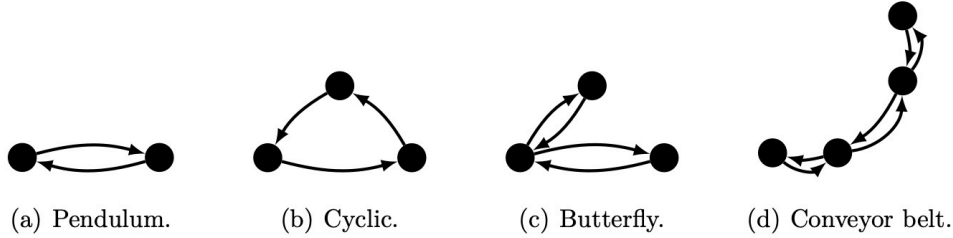
- Line: (1, 2, 3, 4).
 - See the section "Routine Types" below for explanations.
- Number of vessels of different classes
 - Class 1: 1 ship
 - Class 2: 2 ships
 - Class 3: 1 ship

(2) Routine Types

In Stefan Guericke PhD Thesis section 2.3, there are basically 4 types:

- Pendulum: $1 \rightarrow 2 \rightarrow 1$.
 - Expressed by (1, 2).

- Cyclic: $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$.
 - Expressed by $(1, 2, 3)$.
- Butterfly: $1 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 1$.
 - Expressed by $(1, 2, 1, 3)$.
- Conveyor belt: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1$.
 - Expressed by $(1, 2, 3, 4, 3, 2)$.



(3) Restrictions

All these types can be expressed by the sequence of ports with some addition rules:

- Rule 1: the neighbor of every port should be **different**
 - Rule out $(1, 2, 2)$ which is expanded to $1 \rightarrow 2 \rightarrow 2 \rightarrow 1$.
 - Rule out $(1, 2, 1)$ which is expanded to $1 \rightarrow 2 \rightarrow 1 \rightarrow 1$.
- Rule 2 : there must be **at least two segments** between **identical segments**
 - Rule out $(1, 2, 3, 2, 3)$ which is expanded to $1 \rightarrow 2 \rightarrow 3 \rightarrow 2 \rightarrow 3 \rightarrow 1$. In this example, the slot $2 \rightarrow 3$ appears twice, making the line a little bit duplicated.
 - Allow for $(1, 2, 3, 4, 2, 3)$ which is expanded to $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 2 \rightarrow 3 \rightarrow 1$. There are 2 slots between two $2 \rightarrow 3$.
- Rule 3: every port p can be visited **at most n_p times**.
 - Tune the parameter n_p to reduce the size of action spaces
 - For important ports like singapore, n_p can be set high, while for other ports, we can set n_p low.

(3) Vessel

Each vessel has the following attributes

- rank
- capacity (TEU)
- draft (meters)
- daily chartering cost
- bunkering cost function, represented by a linear form $L = (b_0, b_1, b_2)$. Hence, the bunkering cost can be calculated by

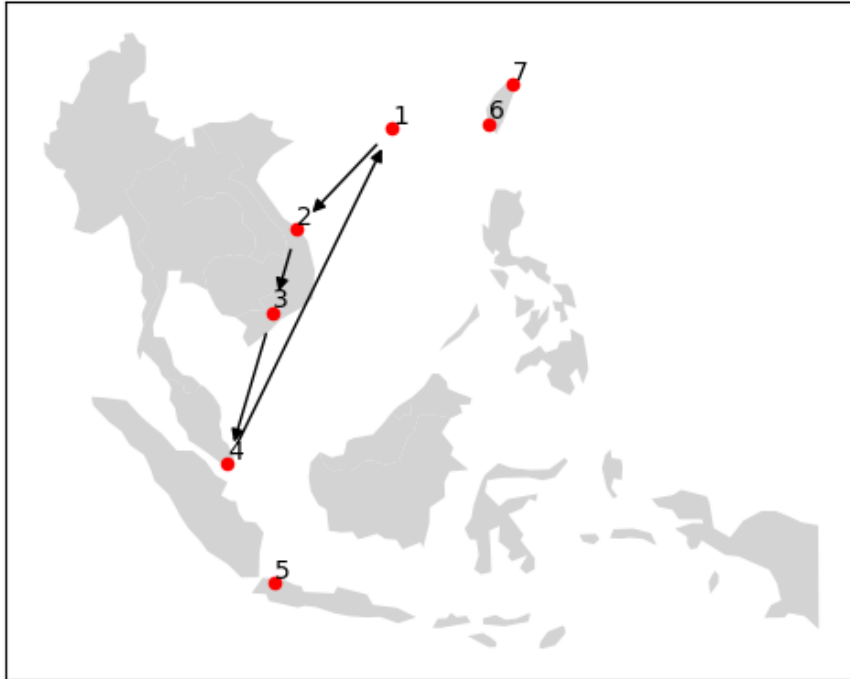
$$\text{Bunkering Cost} = b_0 + b_1 v + b_2 v^2 \quad (4)$$

where v is speed (KTS). Currently, we assume $b_2 = 0$ for simplicity.

(4) A Simple Example:

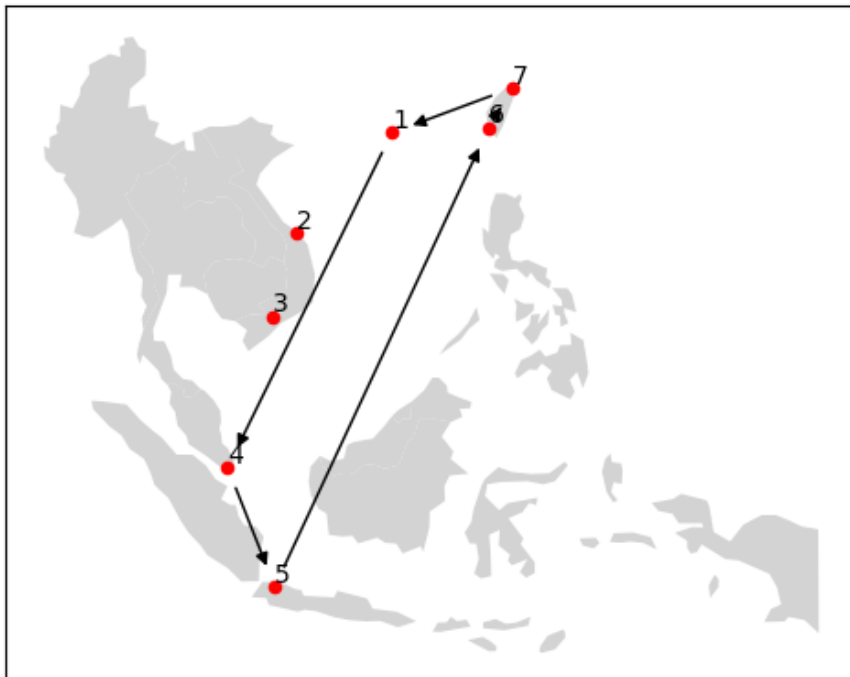
(1) Service 1: Cyclic

- Notation: $T_1 = (1, 2, 3, 4)$.
- Expanded Form $|T_1| = \{(1, 2), (2, 3), (3, 4), (4, 1)\}$.
- Ships: $(1, 2, 1)$.



(2) Service 2: Cyclic

- Notation: $T_2 = (1, 4, 5, 6, 7)$.
- Expanded Form $T_2 = \{(1, 4), (4, 5), (5, 6), (6, 7), (7, 1)\}$.
- Ships: $(1, 2, 1)$.



3. How to Calculate the Total Cost?

3.0. Preparation: Path

Given the services T_1 and T_2 , we can search all paths that connects (o, d) :

- Define $\mathcal{P}_{o,d}$ the set of all connected paths that is not circled
 - Every path $p \in \mathcal{P}_{o,d}$ contains a set of line segments (i, j) in either service $|T_1|$ or service $|T_2|$.
 - A path that connects (o, d) can contains edges of different services.
 - Given the path p , we know the following information
 - Whether a slot (i, j) is included in the path
 - If (i, j) is included, which service provides the connection.

Example: Demand from port $\mathbb{P}_4$ to $\mathbb{P}_1$ can be fulfilled by two paths, either $(4, 1)$ by service 1 or $(4, 5, 6, 7, 1)$ by service 2.

- We use **greedy algorithm** to search all paths for every (o, d) pair.
 - See `_get_paths(...)` method of class `ServiceGraph` in `servicegraph.py` file.

3.1 Optimization Problem

3.1.1 Key Decision Variables

- **Weekly Demand Flow:** $X_{od,p}$
- **Weekly Line Flow:** $Y_{line,edge}$
- **Vessel Number** (integer): $V_{line,r}$
- **Weeks** (binary): $N_{line,w}$

3.1.2 Intermediate Expressions

1). Weekly Transship Amounts

$$Tr_{line,port} = \sum_{\text{paths } p \text{ through port}} X_{od,p} \quad (5)$$

Sum of all $X_{od,p}$ whose path p going through the port. Each Tr element is a linear combination of $X_{od,p}$.

2). Line Capacity

$$C_{line} = \sum_{\text{all vessels}} \text{Capacity of vessels in line} \quad (6)$$

C_{line} is the sum of all vessel's capacity in a line. Each C_{line} element is a linear combination of $Y_{line,edge}$.

3.2 Model

3.2.1 Weekly Transshipment Cost (WTC)

For each *line* and *port*:

$$\text{WTC} = KT \times \frac{Tr_{line,port}}{\sum prods} \quad (7)$$

where KT is hourly transshipment cost of the port, $prods$ denotes productivities of the port for all vessel class, the approximated duration is $\frac{Tr_{line,port}}{\sum prods}$.

$$\sum prods = prods@(V_{line,rank} \geq 0).$$

Optional (Big M constraint):

Given the data, we know that not all vessels can be transshipped.

Hence, we add a constraint by Big M's method:

If $Tr_{line,port} > A$, then there should be a vessel that is smaller than class k , where A is a tuning parameter,

$k = \arg \max_k prods_k$ is the most suitable vessel class for transshipment in this port. More specifically,

$$\begin{aligned} V_{line,0} + V_{line,1} + \dots + V_{line,k} &\geq z \\ Tr_{line,port} - A &\leq z \cdot M \end{aligned} \quad (8)$$

where $z = \mathbb{1}_{Tr_{line,port} > A}$ and A is a tuning parameter.

3.2.2 Weekly Chartering Cost (WCC)

For each $line$ and r (vessel class):

$$WCC = 7 \times KC \times V_{line,r} \quad (9)$$

where KC is daily chartering cost of the vessel class.

3.2.3 Weekly Bunkering Cost (WBC)

For each line, introduce binary speed level variable $KTS_{line,k}$, where k denotes for the level of speed.

Then, we have a one-hot constraint:

$$\sum_k KTS_{line,k} = 1 \quad (10)$$

Weekly bunkering cost:

$$WBC = \sum_{r,k} V_{line,r} \cdot KTS_{line,k} \cdot DCT_{r,k} \cdot 7 \quad (11)$$

where DCR is *daily cost rate* for each vessel class r at speed k ,

$$\frac{\text{distance}}{\text{sailing days}} \approx \text{speed}, \quad 10 \leq \text{speed} \leq 18 \quad (12)$$

There are two options:

- For simplicity, we impose a constraint that

$$DCR_{r,k} = DCR_{r,15} \quad (13)$$

$$KTS_{line,5} = 1 \quad (14)$$

- Optional (Linearization using Big M method):

If we want to optimize the speed, the Big M's method is resorted to linearize this objective.

Let $W_{r,k} = V_{line,r} \cdot KTS_{line,k}$, $M = \sup |V_{line,r}|$:

$$\begin{aligned} W_{r,k} &\leq M \cdot KTS_{line,k} \\ W_{r,k} &\geq -M \cdot KTS_{line,k} \\ W_{r,k} &\leq V_{line,r} + M \cdot (1 - KTS_{line,k}) \\ W_{r,k} &\geq V_{line,r} - M \cdot (1 - KTS_{line,k}) \end{aligned} \quad (15)$$

In this case, the speed constraints are:

$$\begin{aligned} \sum_k KTS_{line,k} (k + 0.5) \cdot 24 \cdot \text{sailing days} &\geq \text{distance} \\ \sum_k KTS_{line,k} (k - 0.5) \cdot 24 \cdot \text{sailing days} &\leq \text{distance} \end{aligned} \quad (16)$$

Note that this constraint is nonlinear:

The approximated time of staying in port for each line is *port staying days*, denoted by *PSD*. Hence,

$$\text{sailing days} = \sum_w \cdot N_{line,w} \cdot 7 - PSD \quad (17)$$

We can linearize the constraints using the same method:

Let $M = \sup |\text{sailing days}|$ and $W_{line,k} = \text{sailing days} \cdot KTS_{line,k}$.

Then, the constraints are transformed to:

$$\begin{aligned} \sum_k W_{line,k} \cdot (k + 0.5) \cdot 24 &\geq \text{distance} \\ \sum_k W_{line,k} \cdot (k - 0.5) \cdot 24 &\leq \text{distance} \\ W_{line,k} &\leq M \cdot KTS_{line,k} \\ W_{line,k} &\geq -M \cdot KTS_{line,k} \\ W_{line,k} &\leq \text{sailing days} + M(1 - KTS_{line,k}) \\ W_{line,k} &\geq \text{sailing days} - M(1 - KTS_{line,k}) \end{aligned} \quad (18)$$

3.2.4 Weekly Port-call Cost (WPC)

From the data, we have the average portcall cost in each port for each type of ship.

$$WPC = \sum_{port,w} \frac{V_{line,r} \cdot C_{port,r} \cdot N_{line,w}}{w} \quad (19)$$

where w is week, $w \in \frac{1}{2}, 1, 2, 3, 4, 5$, r is the vessel rank and $C_{port,r}$ is the portcall cost of the port $port$ and vessel rank r ,

which is a known parameter. The constraint of the week is:

$$\sum_w N_{line,w} = 1 \quad (20)$$

This is also a nonlinear expression, hence, we resort to big M's method:

Let $W_{port,w} = C_{link,port} * N_{line,w}$ and $C_{line,port} = V_{line,r} @ C_{port,r}$, the linear constraints can be equivalently expressed by the auxiliary variable $W_{port,w}$.

$$\begin{aligned} W_{port,w} &\geq M \cdot N_{line,w} \\ W_{port,w} &\leq -M \cdot N_{line,w} \\ W_{port,w} &\geq C + M(1 - N_{line,w}) \\ W_{port,w} &\leq C - M(1 - N_{line,w}) \end{aligned} \quad (21)$$

3.2.5 Other Constraints

1. **Demand fulfillment:** Weekly Demand is not larger than the weekly demand flow for each od pair

$$D_{od} \leq \sum_p X_{od,p} \quad (22)$$

2. **Flow transform:** Weekly line demand flow is not larger than weekly edge flow

$$\sum_{edge \in p} X_{od,p} \leq \sum_{line} Y_{line,edge} \quad (23)$$

3. **Line capacity constraint:** Weekly line flow is not larger than weekly line capacity

$$Y_{line,edge} \leq \frac{C_{line}}{\sum_w N_{line,w} \cdot w} = \sum_w C_{line} \cdot N_{line,w} \cdot \frac{1}{w} \quad (24)$$

(This constraint is nonlinear; linearization via Big M method is implemented in code.)

4. Adjustment of Services: MCTS

The flowchart of the MCTS algorithm with Neural Networks

4.0. Define State, Action and Value Function

- **Service Action.** Suppose a service T contains a sequence of ports. Then
 - Service Action is denoted by $a = (p, l)$ where $p \in \mathcal{I} = \{P1, P2, P3, P4, P5, P6, P7\}$ means a port in the pool of ports and $l \in (0, 1, \dots, |T| - 1)$ means a location related to p .
 - Rules:
 - If $p \notin T$, then **insert** p into T at the location l .
For example, if $T = (P1, P2, P3)$ and $a = (P4, 0)$, then the updated line $T' = (P4, P1, P2, P3)$.
 - If $p \in T$, meaning that there is an element $T.p$ in service. Then, there are two sub-cases:
 - If l is in the neighborhood edges of $T.p$, then **delete** p from T .
For example, if $T = (P1, P2, P3)$ and $a = (P1, 0)$ or $a = (P1, 1)$, then the updated line $T' = (P2, P3)$.
Mathematically, this case occurs when $p = T[l]$ or $p = T[\text{previous of } l]$.
 - If l is not in the neighborhood edges of $T.p$, then **add** p into T at the location l .
For example, if $T = (P1, P2, P3)$ and $a = (P1, 2)$, then the updated line $T' = (P1, P2, P1, P3)$.
Mathematically, this case occurs when $p \neq T[l]$ and $p \neq T[\text{next of } l]$.
 - If service line has only two ports, then cannot delete ports from the line.
 - By these rules, we actually get the broadest variations of lines.
 - Further rules to reduce the action space:
 - Restrict on the limit of the number of visit.
 - ... (See the Section 2.2.3.)
- **Graph Action** is denoted by (a_1, a_2) .
- **Value function.** Suppose we allow for k -step adjustment, then the value of state S is the best reward we can achieve in these adjustments.
 - At each state S , there is a current state reward $R(S)$.
 - The value of the state is

$$V(S_0) = \sum_{k=0}^{\infty} \beta^k R(S_k) \quad (25)$$

where S_k is the updated state by applying action $a_1, \dots, a_{k-1}, a_k$.

4.1. Classical Method: Breadth-First Search (BFS)

(1) Literature

- Victor-Alexandru Darvari, Stephen Hailes and Mirco Musolesi, (2022), Planning spatial networks with Monte Carlo tree search, DOI: [10.1098/rspa.2022.0383](https://doi.org/10.1098/rspa.2022.0383).
- Levente Kocsis and Csaba Szepesvari, Bandit based Monte-Carlo Planning, <http://ggp.stanford.edu/readings/uct.pdf>

- AlphaGo Zero: Starting from scratch (18 October 2017). <https://deepmind.com/blog/article/alphago-zero-starting-scratch>
- Monte-Carlo Tree Search Solver. <https://dke.maastrichtuniversity.nl/m.winands/documents/uctloa.pdf>
- David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. **Mastering the game of go without human knowledge.** nature, 550(7676):354–359, 2017. https://discovery.ucl.ac.uk/id/eprint/10045895/1/agz_unformatted_nature.pdf
- Tong, Z., Liang, Y., Sun, C., Rosenblum, D. S., & Lim, A. (2020). Directed graph convolutional network. arXiv preprint arXiv:2004.13970. <https://arxiv.org/pdf/2004.13970>

(2) Steps: A "Myopic Mind" Algorithm

We are building a search tree. In each tree node, we store the following items:

- sum value
- number of visits n .

Build a root node (default state $T1, T2$):

```
MCT_rootnode = cam.MonteCarloTreeSearchNode(graph)
print(MCT_rootnode.graph.services) # print out the root state
```

```
[[1, 2, 3, 4], [1, 4, 5, 6, 7]]
```

For the root node, currently

- Sum value of root node (q) = value of S_0 .
- Visited number (n) = 1.

Looping for 200 times:

1. Selection.

Find the leaf child with highest UCB score, traversed from the root node and calculate the score of all its children. There are two selections of scores: UCB of child node i is

$$UCT = \frac{q_i}{n_i} + c \sqrt{\frac{\ln(N_i)}{n_i}} \quad (26)$$

or

PUCT, which considers a prior probability

$$PUCT = Q_i + \frac{cP_i\sqrt{N_i}}{1 + n_i} \quad (27)$$

where N_i is the total visited number of the **parent** of the child node i , and $Q_i = q_i/n_i$ if $n_i > 0$ and 0 otherwise.

After picking the best child node, we check whether it has a child node. If yes, then it is not a leaf node and we repeat what we have done to the root node, until we find one that has no children.

Example.

- Round 0: we just find the root node, since it has no child node yet. Then we move to step 2.
- Round 1: we just randomly pick one of all child nodes of the root, since the UCB score of all child nodes are ∞ . For example, we select child node 1. Then we move to step 2.

2. Rollout, Expansion & Backpropagation:

Let's denote by c the selected leaf node.

(i) If the selected leaf node c has **not** been explored ($c.n = 0$), then

- **Rollout.**

Simulate a path of nodes whose length `n_steps` = k from the selected nodes, evaluate their immediate rewards $R(S_i), i = 0, 1, \dots, k$ and update the value of the selected nodes. Remember that

$$V(S_0) = \sum_{k=0}^{\infty} \beta^k R(S_k) \quad (28)$$

we randomly simulate a path of actions $a_1, a_2, \dots, a_k$ (by the output of **policy network**) and generate the path of adjusted states $S_1^c, S_2^c, \dots, S_k^c$.

Then, we calculate their immediate rewards $R(S_k^c)$, and add the maximum one to the **sum value** of the node c .

```
def rollout(self, portgraph: PortGraph, n_steps: int, max_depth: int) -> float:
    """
    Input:
        `n_steps`: the number of adjustments
        `max_depth`: the maximum depth of the whole searching tree
    Work:
        1. Simulate paths of adjustments randomly
        2. Evaluate the value of `self` after `sim_depth` steps of adjustments
    Return:
        self.value
    """
```

◦ Backpropagation.

Update upwardly the value and number of visited of all upper nodes until the root node.

The value of the upper node is updated by

$$\tilde{V}(S^{c.\text{parent}}) \leftarrow \frac{N_k}{1 + N_k} V(S^{c.\text{parent}}) + \frac{\beta}{1 + N_k} [R(S^{c.\text{parent}}) + V(S^c)] \quad (29)$$

where $V^{(k)}(S^{c.\text{parent}})$ is the old **averaged** k -step value. We add $\tilde{V}^{(k)}(S^{c.\text{parent}})$ to the sum of k -step value of the parent node, and add its number of visit by 1.

Meanwhile, in the backpropagation, each updation of values means a **correction** of the previous estimation of the true values.

```
def back_propagate(self, value: float):
    """
    Input:
        `value`: the value of `self`
    Work:
        1. Update the summed value and number of visits of all upper nodes
        2. Update neural network for each upper nodes
    """
```

(ii) If the selected leaf node has been explored ($n \geq 1$), then

◦ Expansion.

Fully expand all its child nodes.

Example.

- Round 0: since the root node c has not been visited, $n = 0$, we will expand the root node and create all child nodes.
- Round 1: since the visited number of child node 1 is zero, we rollout from the child 1.

3. Back to Step 1. (next round)

4.2. Neural Network

(1) Update the policy network

- The objective function

$$J(\theta) = \mathbb{E}_{\pi} [V(s)] = \sum_{a \in \pi} [Q^{\pi}(s, a) \pi_{\theta}(a|s)]$$

- Update the policy network $P(s, a)$ based on the gradient ascent method

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi} [Q^{\pi}(s, a) \nabla_{\theta} \log \pi_{\theta}(a|s)]$$

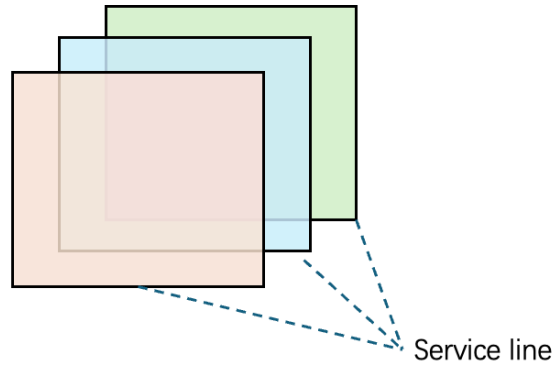
The process is pretty straightforward:

1. Initialize the policy parameter θ at random.
2. Generate one trajectory on policy $\pi_{\theta} : s_1, a_1, r_2, s_2, a_2, \dots, s_T$.
3. For $t = 1, 2, \dots, T$:
 1. Estimate the return $V(s)$;
 2. Update policy parameters: $\theta \leftarrow \theta + \alpha \gamma^t Q^{\pi}(s, a) \nabla_{\theta} \log \pi_{\theta}(a_t|s_t)$

(2) Definition of the action

T is an $n \times n \times m$ matrix, n is the number of ports in the service network (i is the departure port, j is the destination port), m is the number of service lines.

Graph
(three-dimensional matrix, each layer is a service route)



For each $l = 1, \dots, m$:

For each $k = 1, \dots, n; k \neq i, j$:

If $T_{ij}^l = 1, T_{ik}^l = 0$ and $T_{kj}^l = 0; T_{ij}^l \leftarrow 0, T_{ik}^l \leftarrow 1, T_{kj}^l \leftarrow 1$;

if $T_{ij}^l = 0, T_{ik}^l = 1$ and $T_{kj}^l = 1; T_{ij}^l \leftarrow 1, T_{ik}^l \leftarrow 0, T_{kj}^l \leftarrow 0$;

otherwise, T_{ij}^l doesn't update.

For example, T describes a service line (**In-degree Adjacency Matrix**)

Original Line:

1 → 2 → 3 → 1 → 4

T	1	2	3	4
1	0	0	<u>1</u>	<u>1</u>
2	1	0	0	0
3	0	1	0	0
4	<u>1</u>	0	0	0

Add:

Action: Add Port.

1 → 2 → 3 → 1 → 4 → 1

(1, 3) $T_{13} = 1$

⇒ 1) $T_{13} = 0$

2) $T_{1k} = 1, T_{k3} = 1, k = 2.$

	1	2	3	4
1	0	0	0	1
2	1	0	0	0
3	0	1	0	0
4	1	0	0	0

Delete:

Delete "3"

1 → 2 → 3 → 4 → 1

$T_{ij} = 0, T_{kj} = 1, T_{ik} = 1, (i=1, j=2, k=3).$

⇒ $T'_{ij} = 1, T'_{kj} = 0, T'_{ik} = 0.$

	1	2	3	4
1	0	0	0	1
2	1	0	0	0
3	0	0	0	0
4	1	0	0	0

(3) Directed Graph Convolutional Network

The first-order adjacency matrix for directed graphs: A_F :

$$A_F(i, j) = A_{\text{sym}}(i, j) \quad (30)$$

A_{sym} is the symmetric form of the adjacency matrix. A_{sym} represents the service network without directions. The assignment rules for $A_{\text{sym}}(i, j)$ are as follows:

- If there is no service line from port i to port j , and no service line from port j to port i , then $A_{\text{sym}}(i, j) = 0$;
- If there is at least one service line from port i to port j , or if there is at least one service line from port j to port i , then $A_{\text{sym}}(i, j) = 1$;

Note: If the graph has weights, the elements of the adjacency matrix are not limited to 0 and 1, hence the values of A_{sym} are also not confined to 0 and 1.

Then the second-order in-degree adjacency matrix $A_{s_{in}}$ and the second-order out-degree adjacency matrix $A_{s_{out}}$ are proposed as follows:

$$A_{s_{in}} = \frac{\sum_k A_{k,i} A_{k,j}}{\sum_k A_{k,v}} \quad (31)$$

$$A_{s_{out}} = \frac{\sum_k A_{i,k} A_{j,k}}{\sum_k A_{v,k}} \quad (32)$$

In the above formulas, $A_{s_{in}}$ and $A_{s_{out}}$ calculate the second-order connectivity strength between ports by utilizing the structural properties of the graph, where v in the denominator represents all ports that could possibly be connected to ports k . $A_{m,n}$ represents the edge weight between nodes m and n in the original graph.

Three representations for the adjacency of directed graphs are proposed:

- Undirected edge information in the network

$$Z_F = \tilde{D}_F^{-\frac{1}{2}} \tilde{A}_F \tilde{D}_F^{-\frac{1}{2}} X \Theta \quad (33)$$

- Directed edge information in the network

1. In-degree information

$$Z_{s_{in}} = \tilde{D}_{s_{in}}^{-\frac{1}{2}} \tilde{A}_{s_{in}} \tilde{D}_{s_{in}}^{-\frac{1}{2}} X \Theta \quad (34)$$

2. Out-degree information

$$Z_{s_{out}} = \tilde{D}_{s_{out}}^{-\frac{1}{2}} \tilde{A}_{s_{out}} \tilde{D}_{s_{out}}^{-\frac{1}{2}} X \Theta \quad (35)$$

Wherein, Θ are the learned parameters $[W, b]$ in the linear transformation operations (i.e., the linear transformations at each port in GCN without the activation function), X is the feature matrix of the ports.

For each layer of the GCN, $\tilde{A}_x = A_x + I$, $\tilde{D}_x = D_x + I$, $x \in \{F, s_{in}, s_{out}\}$. For the degree matrix D_x of the adjacency matrix $A_x \in \mathbb{R}^{n \times n}$, where $x \in \{F, s_{in}, s_{out}\}$, the calculation rule is (the same as the calculation rule for the degree matrix in GCN):

$$D_x(i, i) = \sum_j A_x(i, j) \quad (36)$$

- The architecture of the Directed Graph Convolutional Network is:

The function to compute the feature matrix $\tilde{Y}$ is defined as $f(X, A)$, $\tilde{Y}$ represents the directed graph:

$$\begin{aligned} \tilde{Y} &= \text{Concat}[ReLU(Z_F), \alpha ReLU(Z_{s_{in}}), \beta ReLU(Z_{s_{out}})] \\ &= \text{Concat}[ReLU(\tilde{D}_F^{-\frac{1}{2}} \tilde{A}_F \tilde{D}_F^{-\frac{1}{2}} X \Theta^{(0)}), \alpha ReLU(\tilde{D}_{s_{in}}^{-\frac{1}{2}} \tilde{A}_{s_{in}} \tilde{D}_{s_{in}}^{-\frac{1}{2}} X \Theta^{(0)}), \beta ReLU(\tilde{D}_{s_{out}}^{-\frac{1}{2}} \tilde{A}_{s_{out}} \tilde{D}_{s_{out}}^{-\frac{1}{2}} X \Theta^{(0)})] \end{aligned} \quad (37)$$

Where, α and β are learnable parameters. Furthermore, if we consider fully connecting them and then use softmax to output the action probability, the model is extended to:

$$\hat{Y} = \text{softmax}(ReLU(\tilde{Y} \Theta^{(1)})) \quad (38)$$

(4) Algorithm framework of the Directed Graph Convolutional Network applied in this network optimization problem